



1. 下列说法中正确的是

- A. 数据是数据元素的基本单位
- B. 数据元素是具有相同性质的数据对象的集合
- C. 数据是带有结构的数据对象的集合
- D. 数据类型是程序设计语言中已实现的数据结构

2. 关于栈的说法正确的是

- A. 栈是限定在表的两端进行插入和删除运算的线性表
- B. 栈的修改是按照先进先出的原则进行的
- C. 栈可以用顺序存储结构和链式存储结构实现
- D. 取栈顶元素运算是将栈顶元素出栈并返回

3. 下列深度为2的广义表是

- A. $LS1=(a)$
- B. $LS2=(a,(b,c))$
- C. $LS3=((),(a),(a,(b,c)))$
- D. $LS4=((a,(b,c)),d)$

4. 多维数组在Pascal语言中是按行优先顺序存储,所以访问二维数组第*i*行第*j*列元素的索引计算方式是

- A. $i * rows + j$
- B. $i * columns + j$
- C. $j * rows + i$
- D. $j * columns + i$

5. 在二叉树的第*i*层($i \geq 1$)上至多有结点

- A. 2 个
- B. 2^{i-1} 个
- C. 2^{i-1}
- D. 2^i 个

6. 下列关于树的说法正确的是

- A. 树可以用广义表形式表示
- B. 非空树可以有零个或一个根结点
- C. 树的结点拥有的子树数称为树的深度
- D. 树中结点的各子树都有序不可交换

7. 关于满二叉树和完全二叉树说法正确的是

- A. 完全二叉树一定是满二叉树
- B. 满二叉树不一定是完全二叉树
- C. 有*n*个结点的完全二叉树深度大于 $\log n$
- D. 满二叉树包括度为0,1和2的结点

8. 对于图的广度优先搜索遍历,采用邻接矩阵和邻接表表示的算法复杂度分别为

- A. $O(n^2)$, $O(n^2)$

02331 数据结构 2025年4月 【广东】

- B. $O(n^2)$, $O(n+e)$
- C. $O(n+e)$, $O(n+e)$
- D. $O(n+e)$, $O(n^2)$

9. 关于二分查找说法正确的是

- A. 二分查找的优点是对表的结构、存储顺序没有要求
- B. 二分查找的过程是递归的, 因此必须用递归算法实现
- C. 二分查找的平均性能远远优于最坏性能
- D. 二分查找的过程可以用一棵二叉树来描述

10. 假设查找每个结点是等概率的, C_i 为找到第 i 个结点所需要比较的次数, 则平均查找长度的计算公式为

- A. $\frac{1}{n} \sum_{i=1}^n C_i$
- B. $\sum_{i=1}^n C_i$
- C. $n \sum_{i=1}^n C_i$
- D. $\sum_{i=1}^n i C_i$

11. 非线性结构的特征是数据结点之间存在着_____或多对多的关系。

12. 数据的运算是定义在数据的_____上的。

13. 单链表中的每个结点的存储地址存放在其_____结点的指针域中。

14. 数组在C语言中是按_____优先顺序存储的。

15. 特殊矩阵中, 非零元素个数远远小于矩阵元素总数的矩阵称为_____。

16. 在描述广义表时又用到了广义表的概念, 所以广义表具有_____的性质。

17. 若一棵树的根结点删除, 就得到该树的子树所构成的_____。

18. 堆排序算法的时间复杂度为_____。

19. 分块查找是两次查找过程, 首先查找_____, 然后在确定的块中进行顺序查找。

20.中序遍历二叉排序树得到的遍历序列是一个递增有序序列,说明对于二叉排序树,若它的左子树非空,则左子树上所有结点的值均_____根结点的值。

21.定义一个循环队列的结点,并写出判断这个队列为满的条件。

22.算术表达式

$(8+2*5)/7$,按照下表的格式写出它转成后缀表达式的过程。(题目中表格略,仅提取问题描述)

23.在应用中如何选择顺序表和链表?

24.输入关键字序列分别为 (50,30,20,70,40,60,80) 和 (70,50,30,80,20,40,60) 时,生成的二叉排序树一样吗?画出它们对应的二叉排序树。

25.输入关键字序列为

(42,18,23,54,67,35),选择基准值为第一个元素,写出快速排序的每一趟结果。

26.阅读算法并回答问题:

```
void f26(int rows, int cols, int A[rows][cols], int B[rows][cols]) {  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < cols; j++) {  
            B[i][j] = A[i][j] + A[j][i];  
        }  
    }  
}
```

(1) 矩阵A如

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

所示,给出矩阵B的计算结果。

(2) 函数f26()的功能是什么?

27.

单链表的结点定义如下：

```
typedef char DataType;
typedef struct node {
    DataType data;
    struct node *next;
} ListNode;
typedef ListNode * LinkList;
```

阅读下列算法，并回答问题：

```
void f27(LinkList head) {
    ListNode *p=head, *r, *s;
    ListNode q;
    while (p!=NULL) {
        s=p; r=p->next;
        while (r!=NULL) {
            if (r->data < s->data) s=r ;
            r=r->next;
        }
        if(s!=p) { q=(*p) ; p->data=s->data; s->data=q.data; }
        p=p->next;
    }
}
```

(1) 函数 f27 () 的功能是什么？

(2) 该算法在单链表上的时间复杂度是什么？与顺序存储结构上的同算法时间复杂度一样吗？

28.函数f28()的功能是根据二叉树的前序遍历序列和中序遍历序列构造二叉树。请在空白处填写适当的语句使函数完整正确。

```
typedef struct Node {
    int val;
    struct Node *left;
    struct Node *right;
} TreeNode;
//函数 createNode() 创建新结点，并返回其地址
TreeNode* createNode(char val);
```

```

TreeNode* f28(char *preOrder, char *inOrder, int first, int last) {
    if (first >= last) return NULL;
    TreeNode *root = ____ (1) ____;
    int i;
    for (i = first; i < last; ++i) {
        if (inOrder[i] == preOrder[first]) break;
    }
    root->left = f28(preOrder, inOrder, ____ (2) ____, i-1);
    root->right = f28(preOrder, inOrder, i+1, ____ (3) ____);
    return root;
}

```

29.函数 f29()的功能是合并两个升序的双链表。请在空白处填写适当的语句使函数完整正确。

```

typedef struct DListNode {
    int val;
    struct DListNode *prev, *next;
} DListNode;

DListNode* createNode(int value); //创建一个新的双链表节点

DListNode* f29(DListNode* list1, DListNode* list2) {
    DListNode dummy = {0, NULL, NULL};
    DListNode* tail = &dummy;
    while (list1 && list2) {
        if (____ (1) ____ ) {
            tail->next = list1; list1->prev = tail; list1 = list1->next;
        } else {
            tail->next = list2; list2->prev = tail; list2 = list2->next;
        }
        ____ (2) ____;
    }
    if (list1) { tail->next = list1; ____ (3) ____; }
    if (list2) { tail->next = list2; list2->prev = tail; }
    return dummy.next;
}

```

30.

图的邻接表存储结构定义如下：

```
#define MaxVertexNum 20

typedef char VertexType;

typedef struct node{           //边表结点
    int adjvex;                //顶点的序号
    struct node * next;        //指向下一条边的指针
}EdgeNode;

typedef struct vnode{          //顶点表结点
    VertexType vertex;         //顶点域
    EdgeNode *link;            //边表头指针
}VNode, Adjlist [MaxVertexNum]; //邻接表

typedef Adjlist ALGraph;      //定义为图类型
```

编写函数 bfs，用广度优先搜索从顶点 1 开始实现图的遍历。函数原型如下：

```
void bfs ( ALGraph G, int n ) //G 为邻接表存储的图，n 是图中结点的数量
```

答案

1.D

解析：数据是信息的载体,是描述客观事物属性的数、字符及所有能输入到计算机中并被程序识别和处理的符号的集合;数据元素是数据的基本单位;数据结构是相互之间存在一种或多种特定关系的数据元素的集合,即数据是带有结构的数据对象的集合;数据类型是程序设计语言中已实现的数据结构,包括数据的表示和操作。

2.C

解析：栈是限定在表的一端进行插入和删除操作的线性表;栈的修改遵循后进先出原则;栈可以采用顺序存储结构(顺序栈)或链式存储结构(链栈)实现;取栈顶元素运算仅返回栈顶元素,不删除元素,出栈操作才会删除栈顶元素。

3.B

解析：广义表的深度是指表中括号的最大嵌套层数。A选项LS1的深度为1(仅有一层括号);B选项LS2的深度为2(外层括号包含原子a和子表(b,c),子表深度为1,总深度为2);C选项LS3的深度为3(子表(a,(b,c))的深度为2,总深度为3);D选项LS4的深度为2(外层括号包含子表(a,(b,c))和原子d,子表深度为2,总深度为2)。但根据广义表深度定义,LS2的子表(b,c)深度为1,故LS2整体深度为2,符合题意。

4.B

解析：行优先存储是按行序依次存储数组元素。假设二维数组有rows行、columns列,第i行第j列元素(行、列索引从0开始)之前有i行,每行有columns个元素,加上当前行的j个元素,总索引为 $i * columns + j$ 。

5.C

解析：根据二叉树性质,第i层($i \geq 1$)上最多有 2^{i-1} 个结点。例如, $i=1$ 时,最多1个结点($2^{1-1}=1$); $i=2$ 时,最多2个结点($2^{2-1}=2$),以此类推。

6.A

解析：树可以用广义表表示,如根结点作为表名,子树作为子表;非空树有且仅有一个根结点;树的结点拥有的子树数称为结点的度,树的深度是树中结点的最大层数;树分为有序树和无序树,有序树的子树有序不可交换,无序树的子树无序可交换,题目未明确说明时不能默认子树有序。

7.C

解析：选项A:完全二叉树是除最后一层外,每层结点数都达到最大,最后一层结点集中在左边。满二叉树是每层结点数都达到最大的二叉树。所以完全二叉树不一定是满二叉树,该项错误。选项B:满二叉树满足完全二叉树的条件,是特殊的完全二叉树,所以满二叉树一定是完全二叉树,该项错误。选项C:根据完全二叉树性质,有 n 个结点的完全二叉树深度为 $\log_2 n + 1$,因为 $\log_2 n + 1 > \log_2 n$,所以该项正确。选项D:满二叉树中每个分支结点都有两个子结点,不存在度为1的结点,只有度为0(叶子结点)和度为2的结点,该项错误。

8.B

解析：邻接矩阵表示图时,广度优先搜索(BFS)需要遍历所有顶点(n 个)和所有边(n^2 条,包括不存在的边),时间复杂度为 $O(n^2)$;邻接表表示图时,BFS遍历所有顶点(n 个)和所有边(e 条,每条边访问一次),时间复杂度为 $O(n+e)$,其中 n 为顶点数, e 为边数。

9.D

解析：二分查找要求数据元素按顺序存储且有序,对表的存储顺序和结构有严格要求;二分查找可以用递归或迭代算法实现;二分查找的平均时间复杂度和最坏时间复杂度均为 $O(\log_2 n)$,性能相近;二分查找的过程可以看作是在一棵二叉判定树上的遍历,每个比较操作对应树中的一个结点。

10.A

解析：平均查找长度(Average Search Length,ASL)是指为确定记录在查找表中的位置,需和给定关键字值进行比较的关键字个数的期望值。当查找每个结点是等概率时,它等于所有结点比较次数之和除以结点总数 n 。选项A中 $\frac{1}{n} \sum_{i=1}^n C_i$,就是先将找到每个结点的比较次数 C_i 累加起来($\sum_{i=1}^n C_i$),再除以结点数 n ,符合平均查找长度的定义。选项B只是比较次数的累加,没有除以结点数,不是平均值;选项C不仅没求平均,还将累加和乘了 n ;选项D中 iC_i 的计算方式不符合平均查找长度的计算逻辑。



11.

答：一对多

12.

答：逻辑结构

13.

答：直接前趋

14.

答：行

15.

答：稀疏矩阵

16.

答：递归

17.

答：森林

18.

答： $O(n \log_2 n)$

19.

答：索引表

20.
答：小于

21.
答：
Typedef struct {
DataType data[QueueSize];
int front, rear;
} SeqQueue;

判断循环队列为满的条件:
(1)Q.rear=Q.front,且用标志位或者元素个数区分空还是满;
(2)或者少用一个元素空间,测试尾指针加1后是否等于头指针。

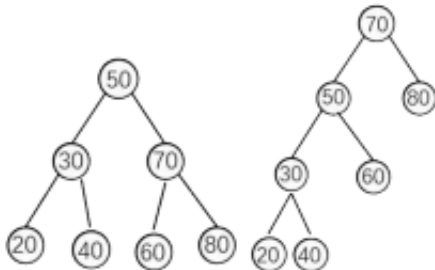
22.
答：

每行 0.5 分，全对 6 分

转换步骤	中缀表达式的读入	运算符栈	后缀表达式
初始	(8 + 2 * 5) / 7 #	#	空
1	8 + 2 * 5) / 7 #	#(	
2	+ 2 * 5) / 7 #	#(	8
3	2 * 5) / 7 #	#(+	8
4	* 5) / 7 #	#(+	8 2
5	5) / 7 #	#(+*	8 2
6	) / 7 #	#(+*	8 2 5
7	/ 7 #	#	8 2 5 * +
8	7 #	#/	8 2 5 * +
9	#	#/	8 2 5 * + 7
10		#	8 2 5 * + 7 /

23.
答：顺序表和链表在应用中根据具体问题的要求和性质来决定。从时间性能考虑,如果对线性表的操作是经常性的查找,以顺序表为宜,如果经常进行的操作是插入和删除,则以链式存储结构为宜。从空间性能考虑,若线性表长度变化较大,以链式存储结构为宜。

24.
答：不一样,对应的二叉排序树分别为



(每个二叉排序树各 2 分)

25.
答：(1) [35 18 23] 42 [67,54]
(2) [23 18] 35 42 [54] 67
(3) [18] 23 35 42 54 67

02331 数据结构 2025年4月 【广东】

(4) 18 23 35 42 54 67

(每得到正确的一轮结果得2分,最多得6分)

26.

答：(1)

$$\begin{bmatrix} 2 & 6 & 10 \\ 6 & 10 & 14 \\ 10 & 14 & 18 \end{bmatrix}$$

(2) 求矩阵A和它的转置矩阵之和。

27.

答：(1)基于单链表的直接选择排序;

(2)时间复杂度是 $O(n^2)$,和顺序存储结构一样。

28.

答：(1)createNode(preOrder[first]) (2)first (3)last

29.

答：(1)list1->val < list2->val

(2)tail = tail->next

(3)list1->prev = tail

30.

答：

```
void bfs(ALGraph G; int n) {  
    CirQueue Q; (1分)  
    int j, k; (1分)  
    InitQueue(&Q); (1分)  
    EdgeNode *p; int visited[20]; (1分)  
    printf("v1→"); visited[1]=1; (1分)  
    EnQueue(&Q,1); (1分)  
    While (!QueueEmpty(&Q)){//算法正确得6分  
        k=DeQueue(&Q); p=G[k].link;  
        while (p!=NULL){  
            j=p->adjvex;  
            if (!visited[j]){  
                printf("v%d→",j); visited[j]=1; EnQueue(&Q, j);  
            }  
            P=p->next;  
        }  
    }  
}
```

答案不唯一，视情况可酌情加分。